

Note: this superceeds OnTheFlyDecomDesignReview.txt; the design has been updated, and details added.

Contents:

Overview
 Requirements
 Design
 1. Use Existing L0_Packets Table
 2. Limited Use of L1 Tables
 3. TelemetryStorageLocation Table
 4. TMDecom Table
 5. Population of the Two New Tables
 6. TDP and Decom Saveset Files
 7. Stored Procedures
 Test Plan
 Open Issues and Q&A
 Performance Results

Overview:

On-the-fly decom means doing the extraction of individual telemetry items from raw packets during data retrieval, not during initial ingest, as is currently done. It means only raw data needs to be stored, not decommed data (with exceptions, see below). Applications which need to fetch telemetry points from the database will do so via an API, rather than by querying database tables directly.

Requirements:

- Greatly reduce current DB storage for IXPE, EMM, GOLD and for future missions. Expect a factor of 10 times smaller. This is the typical inflation figure seen for the size difference between raw data and decommed L1 data, both in files and in DB tables.
- Preserve existing ingest and retrieval performance, within a factor of 2. I.e. it shouldn't take more than twice as long as it currently does to do an ingest or retrieval.
- Provide for certain types of data to still be ingested into, and retrieved from the existing L1 DB tables. This applies to non-packetized data (0158 monitor data, EMM RFTC data, GOLD 1553 data), derived telemetry data, string data, and thinned discrete data. Also support any historical data in L1 we choose to keep accessing from there, etc. as explained in the design. Support an input time range which goes over a date boundary where before the boundary, the data is stored in L1 and after, the data is stored in L0 (or visa-versa).
- Provide an API to access telemetry points via on-the-fly from entire packets stored in L0 DB tables, and to already decommed data in L1 DB tables. Automatically determine which source to get the data from, without user intervention.
 - Save all historical decom maps, with the date/time they came into effect.
 - The API shall support an input retrieval time range which spans different decom maps, i.e. the telemetry item has moved within a packet, or has moved to a different packet, within the time range.
 - The API shall support an input retrieval time range where some of the data is in L0, and some is in L1. The API determines where to get the data from so that applications don't have to do this.
 - Accessing string data and thinned discrete data is not supported by the API. Queries should be done to the appropriate L1 tables, as was done before roll-out of on-the-fly decom.
- The API shall support mission-specific additions and differences, such as EMM's testId or IXPE's retrieval based on the telemetry source file. The API shall expose these such that multi-mission application code needs minimal changes between missions.
- The API shall not preclude access by non-OSW applications like WebTCAD and MatLab tools.
- TDP shall continue to get its decom maps from IDL savesets, not via a live DB query. TDP will determine from the information in 4. whether or not to ingest a given telemetry item into L1.

Design:

The design consists of two new DB tables, and Oracle stored procedures written in PL/SQL which run in the database server.

- Use Existing L0_Packets Table:
 The existing L0_Packets L0 DB table is the data source for the new on-the-fly decom.
 - Initial retrieval testing showed that slightly better performance results when the packet data is stored as a string of hex characters in a VARCHAR, instead of as binary in a BLOB. But that difference (about a 1 sec overhead) is negligible for longer queries where by far most of the time is spent transferring the data from server to client; not retrieving it from the DB.
 - The current mission-specific naming and division of data into multiple L0_Packets tables (e.g. real-time, playback, or by SID) is unchanged for the new L0_Packets table(s).
 - Other, new storage methods are out of scope of this design.
- Limited Use of L1 Tables:
 The traditional L1 DB (TManalog, TMdiscrete, TMstring tables) will continue to store the following data:
 - All derived items.
 Rationale: these would be too hard and/or slow to compute on-the-fly, especially when a derived item is computed from multiple sources in different packets. In addition the config files and/or SPICE calls sometimes used in these computations would give CM issues if the computation was moved to the DB server. IXPE has 24 derived items; EMM has 1000+.
 - All non-packetized data. This data is telemetered as individual measurements in random order; data values are not grouped together in a deterministic order like data in CCSDS packets.
 for IXPE: Malindi Ground Station status items
 for EMM: 0158 Monitor, RFTC status
 for GOLD: individual 1553 telemetry items from SDS files (defined in pseudo-packet apid=11)
 - The CT definitions for such items are typically defined in a pseudo-packet with a special apid, but are otherwise indistinguishable from a downlinked packet.
 - Data in subpackets or variable-length packets where offsets are variable, or where special purpose processing code is required. (GOLD event message packets, EMM SSR log event packets, certain CPRS GSE packets).
 - Telemetry items identified manually as highly popular, or needed for frequent long term plots, if retrieval performance is an issue, can be designated for ingest into and retrieval from the L1 DB.
 - All data before a certain date/time (EMM, GOLD) This could ease the transition by not having to re-ingest all the old data into the new L0_Packets table. In the case of GOLD, this data (the raw record files) isn't even stored on-site, adding to the effort needed to re-ingest it.
 - All strings, at least for this initial version. Rationale:
 - There aren't very many of them, so they don't take up much space.
 - Many strings are telemetered as fixed-length strings, but variable-length strings are also common, like at the end of a variable-length packet or sub-packet. These would be harder to support because their offsets aren't fixed.
 - Telemetry items which are downlinked in multiple packets (apids) might be candidates for storing in L1. By default the on-the-fly decom will query multiple apids for completeness, but it might be slower.

The TMAverage and thinned TMdiscrete tables will continue to be populated and queried from, for missions where these exist. Retrieval software should follow this logic:

- If user requested thinned discretets, query the TMdiscrete_thin L1 table.
- If user requested string data (telemetered or derived), query the TMstring table.

- Otherwise call the on-the-fly decom stored procedure.

3. TelemetryStorageLocation Table:

- This new DB table tells where telemetry is stored for different time ranges: in L0 or L1.
- The data retrieval stored procedure queries it to determine if a requested telemetry item over a given time range is stored in the traditional L1 DB tables or in the new L0_Packets table.
- TDP queries it to determine whether to ingest a given telemetry item into L1.

TelemetryStorageLocation table schema:

```

systemId      NUMBER(5)
apid          NUMBER(5)
tlmId         NUMBER(7)
definitionStart NUMBER(16) ; GPS usec
isInL0        NUMBER(1) ; 1=true, 0=false
isInL1        NUMBER(1) ; 1=true, 0=false

```

Notes:

- If both isInL0 and isInL1 are 1, it means data exists in both L0 and L1 over the ERT range.
- If both are 0, it means the tlm item no longer exists in this packet (apid) after the ERT time.
- The ERT range starts at definitionStart, and continues in effect until the definitionStart of the next row with the same key fields (systemId, apid, tlmId). If there is no next row, the definitionStart is in effect indefinitely (i.e. now).
- The primary, unique key is (systemId, apid, tlmId, definitionStart).
- The query to get the row(s) applicable for the time range a user is asking for is:

```

SELECT definitionStart, isInL0, isInL1, apid
FROM TelemetryStorageLocation WHERE
systemId = systemId_in AND
tlmId = tlmId_in AND
definitionStart <= definitionStopTime AND -- Starts before the desired range ends
definitionStart >= COALESCE((SELECT MAX(definitionStart)
FROM TelemetryStorageLocation WHERE
systemId = systemId_in AND
tlmId = tlmId_in AND
definitionStart <= definitionStartTime), 0)
ORDER BY definitionStart;

```
- The presence of systemId allows a different storage location for the same tlmId depending on system. See discussion in "Population of the Two New Tables".
- stopERT is not included, because it would complicate adding new rows. The previous row for a tlmId would have to have its stopERT updated to the new row's startERT whenever a new row was added for that tlmId. The table would be harder to manage and delete from, as inconsistencies in start/stopERT could arise.
- apid is included, so that telemetry values for the same item downlinked in different packets (apids) can have different storage locations. For example, a telemetry item downlinked in a short critical HK packet and a long HK packet could have different storage locations.
- For derived items and non-packetized items, no rows exist (see Population of the Two Tables).

4. TMDecom Table:

- This DB table stores historical decom maps for use by the on-the-fly decom.
- There is only one table. It does not have different instances in different schemas, for missions where different schemas are used for different systemIds.

TMDecom table schema:

```

systemId      NUMBER(5) ; allows for the same tlmId in different systems to be decommed differently
apid          NUMBER(5) ; identifies the apid of the packet this row applies to
tlmId         NUMBER(7) ; identifies the telemetry item
definitionStart NUMBER(16) ; the time this row went into effect Same format as ERT in the L0 & L1 tables.
dataType      CHAR(1) ; see CT DB User's Guide for values
startBit      NUMBER ; Zero-based bit offset from the start of the CCSDS packet header
length        NUMBER ; number of bits for the telemetry item

```

If startBit is -1, it means the after the ERT time, tlm item no longer exists in this packet (apid), at least not at the previous location, with this size and dataType.

Notes:

- The unique index is: (systemId, apid, tlmId, definitionStart)
- The query to get the row(s) applicable for the time range a user is asking for is:

```

SELECT * FROM TMDecom WHERE
systemId = systemId_in AND
apid = TSLRows(i).apid AND
tlmId = tlmId_in AND
definitionStart <= TMDQueryStopTime AND
definitionStart >= COALESCE((SELECT MAX(definitionStart) FROM TMDecom WHERE
systemId = systemId_in AND
tlmId = tlmId_in AND
definitionStart <= TMDQueryStartTime), 0)
ORDER BY definitionStart;

```
- startBit is not included, because we don't support the same item twice at different locations in the same packet.
- The on-the-fly decom usually uses the input ERT for the queries to TelemetryStorageLocation and TMDecom. If the caller only provides a SCT (spacecraft time converted to UTC) range, then the queries use that as a ERT range.
- If the user specifies an EMM testId, and no time range(s), the on-the-fly decom translates the testId into a ERT range.
- If user wants playback data, ERT may not be accurate, even if provided.
- In this and other cases, to get correct results the user may have to explicitly specify the date/time of the definitions they want to use. This option is supported. The selected decom map will be used for on-the-fly decom of data within the user's start/stop times.
- The case of a telemetry item changing name would be dealt with by changing the tlmId-to-externalElement/itemName mapping in the TelemetryIDRegister table, as part of the release process.
- This is the same as the current capability, no change.

5. Population of the Two New Tables:

- The TelemetryStorageLocation table does not contain derived and non-packetized telemetry items.
- If the stored procedure is called with a tlmId which isn't in TelemetryStorageLocation, then it's assumed to be stored in L1.
- Rationale: this makes the system more robust, and less complicated by derived items. There are myriad ways to define derived items:
 - They may be defined in one or more pseudo-packets: packet definitions which don't represent a downlinked packet, but are just a convenient way to define telemetry items, and have an otherwise unused apid.
 - They may be present in the TelemetryDerivations table.
 - They may have been put in the TelemetryDerivations table, but have since been deleted by a new CT release which didn't take them into account (e.g. IXPE, 25 total derived items)
- The TMDecom table contains *only* rows for the numeric packetized tlmIds for a mission, regardless of their current L0 or L1 storage location. The TMDecom table does not contain derived items, non-packetized items, string items, or byte array items.

The process of initially populating the two new tables, then updating them for each new FSW release, is:

- Update the Oracle CT DB with new definitions. This can be done for a single systemId, or for all of them. This is typically done for each new FSW release.
- Create new decom savesets for TDP to use. This is the same as the current process. There are different saveset files for each systemId. The saveset is generated from the Oracle CT DB.
- Run updateOnTheFlyDecomTables for the desired systemId(s). This is stand-alone IDL program for initially populating the two tables, and for updating them. It is mostly generic code, with mission-specific parts.
 - Input: CT DB, optional startERT to tag each new row with instead of current system time (which is the default)
 - Output: A sql script which inserts into the TelemetryStorageLocation and TMDecom tables. Run it using sqlplus.
 - Usage: For initial population of empty tables, pass a startERT equal to the first ERT that exists in the L0 or L1 DB's. For subsequent updates, this should be run *after* the CT DB has been updated.
 - It needs mission-specific, hard-coded logic to determine the storage locations according to the criteria listed in 2. above.
 - It only does inserts, never deletes or updates.
 - It does *not* add identical new rows with a new time-tag for each FSW release.
 - It adds a new row when:
 - There is a decom change (apid, offset, length, dataType)
 - A change to a TelemetryStorageLocation row which resulted from a change in the program's logic

- or hard-coded cases.
- Rationale: this results in fewer rows for the on-the-fly decom to process.
- The definitionStart of all new rows created in both tables should be identical.
- Population of the two tables and creation of the saveset(s) should use the same date/time in new rows.
- d. The TelemetryStorageLocation table may be updated independently of the TMdecom table in special cases.
- IXPE scenario:
 - This scenario is only necessary if on-the-fly performance is an issue; it keeps data in L1 for a few months before deleting it; data older than a few months must be accessed from L0.
 - Partition the L1 tables (TManalog_SID1, TMdiscrete_unthinned_SID1) into monthly partitions.
 - TDP always ingests telemetry into both L1 and L0 (no change from current behavior).
 - Every month, drop the oldest 1 month partition (e.g. the one from 3 months ago):
 - Copy all the points from the 25 derived items (and any other high demand items) from the old partition to a new, empty partition with the same time range.
 - Delete the old, fully populated partition, so the storage is freed. The old items are now only accessible from L0, and derived items are the only items in the old partition.
 - Update (delete and insert because descriptionStart is a key field) the TelemetryStorageLocation table rows for all the normal telemetry items:
 - row 1 descriptionStart (launch date)
 - isInL0 1
 - isInL1 0
 - row 2 descriptionStart (now - 3 months) --> change to: (now - 2 months) i.e. add 1 months
 - isInL0 1
 - isInL1 1
 - I made a picture of this on paper.
 - If there are any high demand items, their TelemetryStorageLocation rows look like this.
 - and don't ever change:
 - descriptionStart (launch date)
 - isInL0 1
 - isInL1 1
 - The 25 derived items don't have rows in TelemetryStorageLocation.
- The mission-specific logic described in 5c. is as follows:
 - IXPE: No special cases are needed.
 - Apids 32,33,34 are created by OIS to package Malindi non-packetized telemetry. These packets are just like spacecraft packets: they are defined in the CT DB, OIS saves them to a raw record file, and they are ingested by TDP into the L0_Packets table, where they can be on-the-fly decommed.
 - Their definitions are in the TDP decom saveset.
 - Derived items were defined via TelemetryDerivations, with no packet definitions.
 - So updateOnTheFlyDecomTables will not create any rows for these items.
 - The OTFD stored procedure will see that the tlmId for one of these derived items has no row in TelemetryStorageLocation, and will then assume it's stored in L1, as desired.
 - For TDP, the items in apids 32,33,34 are in the decom saveset and in TelemetryStorageLocation with isInL1=0, so TDP will not ingest them into L1, as desired.
 - GOLD: GOLD has a single systemId.
 - The decom definitions are not expected to ever change.
 - The TelemetryStorageLocation table will probably designate that all analog and discrete data after the deployment date of OTFD is in L0_Packets, except the raw telemetry items in event message subpackets, and spacecraft telemetry, which both stay in L1.
 - Add DB_OTFD_ADD_ISINL1_FLAG to gold_db.pro; call DB_CONNECT from gold_tdp.pro
 - Spacecraft telemetry is defined in the pseudo-packet with apid=11, so the logic in updateOnTheFlyDecomTables needs to exclude apid=11. Then there will be no rows in TelemetryStorageLocation for spacecraft items, and the OTFD stored procedure will assume they're stored in L1, as desired.
 - The apid=4 event subpacket telemetry items are not in the decom saveset, so updateOnTheFlyDecomTables will not put rows in TelemetryStorageLocation. The OTFD stored procedure will see that a tlmId for one of these subpacket items has no row in TelemetryStorageLocation, and will then assume it's stored in L1, as desired.
 - For TDP, the apid=4 subpacket processing code does not call decom_tlm, but rather decomms the points manually itself and writes them to the analog bcp file.
 - CPRS: The TelemetryStorageLocation table will designate that all downlinked analog and discrete items in top-level packets is in L0_Packets. Telemetry items in subpackets is not handled by default by updateOnTheFlyDecomTables, because they're not in the main decom map, so no rows will exist in TelemetryStorageLocation. Thus the OTFD stored procedure will see that a tlmId for one of these subpacket items has no row in TelemetryStorageLocation, and will then assume it's stored in L1, as desired.
 - For TDP:
 - Add DB_OTFD_ADD_ISINL1_FLAG to cprs_db.pro; call DB_CONNECT from cprs_tdp.pro
 - The code for the one derived item writes to the TManalog sqldr file itself, as desired.
 - The subpacket processing code calls decom_tlm, with a subpacket decom map that came from the decom saveset file. This code needs to manually augment its decom structure with the new tag isInL1 set to 1, before it calls decom_tlm. Then decom_tlm will write the decommed points to the TManalog sqldr file.
 - For all normal telemetry items, a new DB_OTFD_ADD_ISINL1_FLAG IDL procedure in cprs_db.pro should be the same as IXPE, querying TelemetryStorageLocation in real-time for the isInL1 flag, and adding it to all the input decom structures. This will cause TDP to write decommed telemetry points to the L1 sqldr files if TelemetryStorageLocation.isInL1=1, and not write them if isInL1=0. This should be called by
 - Aside: CPRS has 3 or so systemIds: FLIGHT, FLIGHT_IT, EM. Like TSIS, CPRS uses the same tlmIds in these systems. CPRS does not use externalElement prefixes to generate different tlmIds per-systemId, unlike WMS, EMM, and GOESR which do.
 - TSIS2: Currently has no subpacket code, but CT DB does have similar subpackets as TSIS. See tsis2_tdp_l1.pro. The TelemetryStorageLocation table will probably designate that all downlinked analog and discrete items should be in L0_Packets. However TDP_L0_include_apids.txt has a list similar to TSIS, so not all packets are being ingested into L0_Packets.
 - Add DB_OTFD_ADD_ISINL1_FLAG to tsis2_db.pro; call DB_CONNECT from tsis2_tdp.pro
 - Spacecraft telemetry is ingested from a WOD file into L1 by read5CTelemetry.py
 - The spacecraft telemetry items are defined in apid=9999.
 - updateOnTheFlyDecomTables can do the default for all packets except for apid=9999, where it should should set isInL1=1. Thus the OTFD stored procedure will go to L1 for spacecraft items, and do OTFD from L0_Packets for all other items. At least starting on the date the OTFD tables are initially populated, and all apids have been added to TDP_L0_include_apids.txt.
 - EMM: Contents are TBD, but will probably be similar to GOLD: starting at the release date, most packetized data will be designated to be stored only in L0.
 - The table contents change with each new FSW release. Typically tlmIds are moved around within and between packets, new tlmIds are added, and some tlmIds are deleted. After the CT DB has been updated, the release process will run the program to update the tables and generate new IDL savesets. The startERT in the tables, and the date/time in the saveset filenames reflect when the new build was generated from the CT DB, not when FSW was loaded onto the spacecraft, Flatsat or instrument.
 - If necessary, at a later time, the startERT date in the table(s), and the date/times in the saveset filenames can be made more accurate. This hasn't been necessary with the old system, because people expect bad data to exist around the time of loading a new version of FSW. Therefore it probably won't be necessary with the new system.
 - A similar issue exists if we decide to back-fill L0_Packets with old data prior to the release date. We would insert historical decom maps, using CT DB savesets as the source. The first step would be to make the startERT times more accurate, by finding exactly when FSW was loaded, or when OTIS telemetry was changed.
 - To find when and which packets changed lengths, plot the lengths from ingested telemetry, and determine the exact date/time when a length changed. Probably every release has a packet which changes length.
- Discussion:
 - a. A separate decom saveset file exists for each systemId. This is unchanged from the current design.
 - b. TelemetryStorageLocation:
 - If a component is moved to a different system (e.g. Flight model is moved from a test system to the spacecraft), a new row may be needed with the new systemId. Normally a given tlmId will have a fixed storage location, regardless of systemId. But the design allows for flexibility here:
 - On EMM, data (the same tlmIds) taken when the instrument is on the spacecraft (system=AIT) or stand-alone (system=EXT)EMUS(EMIRS) could be stored in different locations.
 - On TSIS or CPRS, the EM and FLIGHT models could store data in different locations, even though the tlmIds are the same. (These missions use the same tlmIds for similar telemetry items in EM and FLIGHT models).
 - c. TMdecom:
 - The presence of systemId in the table schema allows a EM and a FLIGHT model to have the same tlmId in different packets (apids), or in different locations in the same apid.
- 6. TDP and Decom Saveset Files:
 - TDP = Telemetry Data Processing, ingests data from a raw telemetry file into the L0 & L1 databases.
 - L1 ingest currently uses a decom saveset file which contains only one historical version of the decom maps, not all of them like the TMdecom table. TDP gets all its decom information from the most recent saveset file;

- it does not query the TMdecom table; that is only used by on-the-fly decom on the retrieval side.
- TDP decoms all tlmIds specified in the decom saveset, regardless of their L0 or L1 storage locations. It does this because it needs to pass decommed items to DPC for checking, and it also uses decommed values in derived item calculations. The time used to decom every tlmId is small compared to the total ingest time, so this is not a performance issue.
- When TDP does L1 processing on a file of packets (all with the same apid), it queries the TelemetryStorageLocation where apid=x and isInL1=1. TDP then augments the saveset's structure with the isInL1 flag for every tlmId. This tells TDP (decom_tlm.pro) whether to write the decommed data for a given tlmId to the L1 sqldr file. Q: Is it OK to use the most recent TelemetryStorageLocation row for each tlmId? Or should it query for the nearest row before the decom saveset's time-stamp?
- A:
 - The current method for reprocessing old data by specifying the saveset file to use remains exactly the same.

TDP runtime details:

- TDP does a live query to the TelemetryStorageLocation table, and adds a do_L1_ingest flag to the in-memory decom maps for each tlmId, according to the latest TelemetryStorageLocation row's isInL1 value for the tlmId.
- The multi-mission decom_tlm procedure gets passed a complete decom map, no narrowing is done (except for apid). TDP decoms everything, because it needs to pass decommed data to DPC and to derived item creation routines. When decom_tlm is ready to write to the sqldr file, it checks the do_L1_ingest flag in the current decom record, and doesn't write it if do_L1_ingest=0.
- The above query uses the latest TelemetryStorageLocation rows by default. A new input, 'TSL_date' to tdp and ingest_data could change this to use the latest rows as of a certain date, or to not use the table at all, and ingest everything.
TSL_date could be:
 - 'filename_timestamp' : uses time-stamp from file name; may be partly mission-specific to handle non-OIS raw record filename time-stamps.
 - 'yyyy/ddd-hh:mm:ss' : (and format variants)
 - 'ignore' : ingests all L1 data
 This is not a requirement, and has not yet been implemented...FIXME!

7. Stored Procedures:

a. Packages

The code is divided into multi-mission and mission-specific, Oracle-specific PL/SQL code:

onTheFlyDecom.pks	Package spec for multi-mission code.
onTheFlyDecom.pkb	Package body for multi-mission code.
onTheFlyDecomMissionSpecific.pks	Package spec for mission-specific code.
onTheFlyDecomMissionSpecificIXPE.pkb	Package body for IXPE-specific code.
onTheFlyDecomMissionSpecificEMM.pkb	Package body for EMM-specific code.

The stored procedures and global temporary tables are available to the <mission>.tcad user, e.g. ixpe_tcad, emm_tcad.

b. Public Stored Procedures:

When calling these, prefix the name with onTheFlyDecom package, e.g. onTheFlyDecom.getVersion.

PROCEDURE getVersion

Writes two rows to the onTheFlyDecom_errors table, giving the current multi-mission code version and the current mission-specific code version. Example contents:

```

MESSAGE
-----
INFO multimission version: 1.0
INFO mission-specific version: IXPE 1.0

```

PROCEDURE setOption(optionName IN VARCHAR2, optionValue IN VARCHAR2)

Sets an option to control the main selectNumericTlm procedure. Current options are:

optionName	optionValue
DEBUGLEVEL	0=off, 1=slightly verbose, 2=very verbose
APIDS	"xx[,yy[,zz]]" etc. If a telemetry item occurs in more than one apid, this overrides the default (get data from all apids) by specifying one or more apids in a comma-separated list.
DECOMAPTIMEGPS	nnnnnn - Use the decom map in effect at this time. Specify as integer GPS microseconds. Overrides the default of using the user's input time range to determine which decom map(s) to use.
These two are for test purposes:	
FORCEISINL0	0=change the isInL0 column value queried from the TelemetryStorageLocation to 0.
	1=change the isInL0 column value queried from the TelemetryStorageLocation to 1.
FORCEISINL1	0=change the isInL1 column value queried from the TelemetryStorageLocation to 0.
	1=change the isInL1 column value queried from the TelemetryStorageLocation to 1.
IXPE only:	
TLNFILENAME	Only query data that came from the specified telemetry file name.
EMM only:	
TESTID	Only query data with this testId.
VCS	1=only return real-time data, 2=only return playback data, 3=return both

If you specify a non-existent option, the available options will be listed in a message in the onTheFlyDecom_errors table.

PROCEDURE clearOption(optionName VARCHAR2)

Restores the default of a previously set option. Same optionName values as above, plus the "ALL" optionName, which will restore the defaults for all options.

PROCEDURE selectNumericTlm(systemId_in IN NUMBER, tlmId_in IN NUMBER, startERT_in IN NUMBER, stopERT_in IN NUMBER, startSCT_in IN NUMBER, stopSCT_in IN NUMBER)

systemId_in - The systemId to retrieve data from. See the mission's SystemsDefinition table for values.
 tlmId_in - The tlmId of the desired telemetry item, from the TelemetryItemDefinition table.
 start/stopERT_in - Specify the ERT range of the desired telemetry, as GPS microsecond values. Otherwise set them to -1.
 start/stopSCT_in - Specify the spacecraft time range of the desired telemetry, as GPS microsecond values. Otherwise set them to -1.

c. Stored Procedure Usage:

Applications should first call setOption, if needed for special cases. This is usually not needed. Applications always call the main stored procedure: onTheFlyDecom.selectNumericTlm, which gets the desired telemetry data into a temporary table. Then they select the data (SCT, ERT, Value) from the temporary table, with an order by clause. The global temporary table is a named table: "onTheFlyDecom_results" with this table schema:

```

SCT      NUMBER(16),
ERT      NUMBER(16),
VALUE    NUMBER

```

A separate instance exists for each database connection.

Then the application should query the "onTheFlyDecom_errors" table. If no errors occurred, this table will be empty. If errors occurred, or if the user set the debugLevel option to > 0, then this table will contain one or more strings. The table schema is:

```

SEQUENCE  NUMBER,
MESSAGE   VARCHAR2(500),
OCCURRENCES  NUMBER

```

Each MESSAGE string starts with "INFO", "WARNING" or "ERROR", followed by the message string. The query should contain an "order by SEQUENCE" clause, so the error messages are in time order. If the exact same error occurred multiple times, only one row will exist, and the OCCURRENCES column gives the number of times that error occurred. But if the user set the debugLevel option to > 0, all occurrences will exist as separate rows, in time order, and OCCURRENCES will be 1 for all rows.

Each database connection has its own independent instance of the global temporary tables and the stored procedures.

Note: for string data, applications should query the L1 TMstring table as they do now. String data is not supported by the on-the-fly decom stored procedure.

d. Stored Procedure Details:

selectNumericTlm queries a L1 table, or does on-the-fly decom from a L0_Packets table, or both, depending on what's in the TelemetryStorageLocation table. The application does not need logic to do this.

Steps:

- The selectNumericTlm procedure queries the TelemetryStorageLocation table using the input systemId, tlmId and ERT and/or SCT time-range. See 4. above for note about using ERT.
 - Then it loops through each row in the results: (startERT, apid, location).
 - Each row has a ERT start/stop range, and these ranges are contiguous with each other over the entire time range.
- If the location is L1, it queries the appropriate L1 table, and adds the results in a temporary table.
- If the location is L0, it queries the TMdecom table. If the query returns more than one row, it sorts them into time order, and the following is done in an inner loop.
 - For each combination of location and decom location, it calls a private on-the-fly decom stored procedure, which adds the results to the temporary table.
 - If there was a single packet definition over the time range, i.e. one TMdecom row, there will be just one call.
 - This is what's expected in flight, when packet definitions are unchanging.
- The application can now select the data out of the temporary table, with an order by clause.
 - It can do this by creating a cursor, then doing multiple fetches from the cursor, until it has read all the data from the temporary table.
- The application should finish fetching from one tlmId before making a selectNumericTlm call for a new tlmId.
 - The temporary table contents are deleted by selectNumericTlm before it inserts new data for a new tlmId.
- If an item is stored in L0 and is downlinked in multiple packets, the default is to get data from all apids and combine the results. An optional apid or list of apids to get the data from can be provided by the application or user.
- The actual on-the-fly decom processing is as follows, given a single set of (apid, offset, length, dataType).
 - It determines from the offset and length, the start location and minimum number of hex characters needed to include the bits of the telemetry item.
 - It queries the L0_Packets table, using DBMS_LOB.SUBSTR extract the data from each row's BLOB.
 - It includes a criteria of: L0_Packets.length >= (offset + length) as a guard, in case the packet ends before the end of the requested substring. Each row of the query result is an array of hex characters.
 - It converts each row of hex chars to binary bytes.
 - For a numeric type, it masks and/or shifts the bytes to extract the bits of the telemetry value into an appropriate native data type.
 - It writes a row containing the ERT, SCT and Value as a NUMBER to the temporary table.

Test Plan:

1. Comprehensive unit tests exist for the core function which decoms numeric data types from a hex string.
2. Performance and completeness testing was/is done by using data which exists in both L0 and L1 databases.
 - Initial testing was done manually in sqlplus, calling the stored procedures directly.
 - Most testing was done after implementing the stored procedure calls in the DB_ROUTINES interface, and using retrieve_eng to write various test scripts in IDL.
 - Performance testing was done using the IDL profiler, comparing the performance with the old L1 code without the OTFD procedure calls. Results:
 - OTFD code from the L0_Packets table takes 1.1 to 1.5 times the old L1 code time.
 - OTFD code querying from L1 tables takes about the same amount of time as the old L1 code.
 - Typical run times for these results were 1 minute (retrieving 10 days of 1 Hz data)
 - A wide range of telemetry items were used: items near the beginning of packets, near the end of packets, with various bit offsets and lengths, both integer types and telemetered 64-bit floats.
 - The same tests were repeated several times on different days and performance results compared.
 - Correctness was verified by comparing the output data from retrieve_eng (time-stamps and values) from the 3 methods.
 - Floating point values were often different, but never by more than 1.0e-16.
 - Time-stamps and integer values were always identical.

Open Issues and Q&A:

- Q: The MAVEN L1 DB tables store SCLK instead of GPS, because the SPICE clock kernel is not updated for the data until many days after TDP ingests the data. What to do if another mission is like this?
- A: TBD
- Presumably the times the application passes to the on-the-fly decom would be SCLK times instead of GPS times, but the on-the-fly decom may not need to know the difference.
- Q: Which telemetry items occur in more than one packet?
- A: On EMM, the following items occur in more than one packet:
SSR NVMATA00..63, SSR NVMINDEX, EMUS (numerous), SIM RwxMODE. The 3 EMUS packets are apid=1101..1103.
- Q: What to do if dataType has changed between numeric and string over the input time range?
- A: Return an appropriate error to the application.
- Changes between signed integer, unsigned integer and float are handled by the on-the-fly decom.
- Changes between discrete and analog are also handled, but the application probably needs to handle it too.
- Q: David Gathright has a TSIS-1 use case where in TRAD they do a query for a bunch of points with a where clause which specifies the value of a different point. I.e. the other points are only valid when the different point has a certain value. Furthermore the nature of the points is very general, so designating a small set of points to be stored in L1 may not be sufficient.
- David noted that DBA Brian found that when this ran it brought the database to its knees.
- A: A small set of points could be designated for L1 storage (max around a few hundred).
- Or this could be done at the application level. I.e. the query for the dependent points could be done after another query had returned the controlling points.

Performance Results:

12/13/18

- I implemented the query and on-the-fly decom part of the design for EMM in DB_ROUTINES (used by XTCAD and retrieve_eng). This did everything in SQL. No PL/SQL.
- The IDL profiler shows the method calling dbms_lob.substr from L0_Packets takes almost 100x longer than the L1 method.
- All runs were on emm-dev1, unless otherwise noted.
 - When the query for a single telemetry item returns ~60K+ points, typical results were:
 - L0 took 21 sec, L1 took 0.3 sec. The variation in timing between multiple runs was not significant.
 - Repeating the same query sometimes only slightly increased L1 performance, but other times increased it dramatically (expect DB server to cache the data). All runs used with different tlmIds to avoid this.
 - Time spent was almost all retrieving results from the cursor, not in creating the cursor, nor in the hex conversions or bit manipulations in IDL.

12/17/18

- I implemented storing the packet as a hex string in a single VARCHAR column, and retrieving with substr in the sql query. Ingest was not noticeably slower than ingest into a BLOB.
- When run on 1 day, retrieval was 5x slower. One day is about 20K rows for this test data.
- When run on 3 days, it was 13x slower: 19.8 sec L0, 1.4 sec L1.
- Another 3 day run was 13x slower: 20.7 sec L0, 1.5 sec L1.
- Each run used a different telemetry item to avoid L1 caching issues (but not L0 caching issues...)

12/17/18

- I implemented retrieving the entire hex string (no substr call).
- When run on 3 days, it was 15x slower: 21.7 sec L0, 1.4 sec L1.
- Another 3 day run: 16x slower: 20.6 sec L0, 1.3 sec L1.

01/04/19

- I repeated the above tests retrieving entire hex string (no substr call)
- When run on 3 days, it was 9x slower: 22.5 sec L0, 2.56 sec L1.
- Another run: 9x slower: 22.2 sec L0, 2.54 sec L1.

01/04/19

- After Brian updated statistics and updated parallel on emm_schema03.L0_Packets_VARCHAR, and I added the parallel hint to the select query:
- When run on 3 days, it was 8.5x slower: 21.8 sec L0, 2.57 sec L1.
- Another run: 10x slower: 22.5 sec L0, 2.26 sec L1.

01/14/19

- Another run: 8x slower: 21.4 sec L0, 2.55 sec L1
- But Brian reports 2 sec for the query:
- ```
select /*+ PARALLEL */ SCT_GPS_USEC, ERT, packet from EMM_schema03.L0_Packets_VARCHAR
```

where testId != 999999 and apid = 0 and ERT between 1228096818000000 and 1228348818000000  
order by SCT\_GPS\_USEC, ERT;  
I ran this in sqlplus in an emacs shell, it took 1 min 12 sec  
I ran this in sqlplus, redirecting the query from a file and output to a file, took 6 sec.

I changed the L0 code to retrieve in blocks of 5000, this seems to have fixed it:  
2x faster: 1.08 sec L0, 2.29 sec L1

11/01/22

The new PL/SQL code is much faster than previous tests, which used straight SQL.  
For large queries (10+ seconds) overall performance is the same for OTFD vs. L1,  
because the time is dominated by retrieving the data from the temporary table or  
the L1 table. For short queries, OTFD has ca 0.7 sec penalty of overhead;  
total short query times are typically 0.3 sec for L1, 1.0 sec for OTFD.

08/03/23

In ixpe-db-dev using retrieve\_eng, the OTFD method takes 1.1 to 1.6 the time of the L1 method.  
So a little slower, but less than twice as slow. This is with the slow version 12 Oracle client  
libraries (see DB-2464).

09/11/23

With the newly deployed version 19 Oracle client libraries, OTFD runs in half the time it used to.  
L1 runs even faster.